

Python Programming
Unit 02 – Lecture 01 Notes
Lists Fundamentals and Comprehensions

Tofik Ali

February 20, 2026

Contents

1	Lecture Overview	2
2	Core Concepts	2
2.1	What Is a List?	2
2.2	Creating Lists (Common Patterns)	2
2.3	Indexing and Negative Indexing	2
2.4	Slicing (start:stop:step)	3
2.5	List Methods: Mutating vs Returning New	3
2.6	List Comprehensions (Filter + Transform)	4
2.7	Nested Lists (2D Data)	4
3	Common Pitfalls (and How to Avoid Them)	5
3.1	Aliasing vs Copy	5
3.2	Modifying While Iterating	5
4	Demo Walkthrough: Student Marks Analyzer	5
4.1	What the Demo Is Teaching	5
4.2	Suggested Run Steps	5
4.3	Extensions (Optional)	5
5	Quiz and Solutions	6
5.1	Checkpoint 1 (Mutating vs Returning New)	6
5.2	Checkpoint 2 (Comprehension)	6
5.3	Checkpoint 3 (Filtering vs <code>if-else</code>)	6
5.4	Think-Pair-Share (Readability)	6
5.5	Exit Question (Slicing)	6
6	Practice Exercises (With Solutions)	7
6.1	Exercise 1: Remove Duplicates (Keep Order)	7
6.2	Exercise 2: Top-3 Without Mutating the Input	7
6.3	Exercise 3: 2D Sum	7
6.4	Exercise 4: Create and Display a List	7
6.5	Exercise 5: Remove All Even Numbers	7

6.6	Exercise 6: Squares from 1 to 10 (Comprehension)	8
6.7	Exercise 7: Second Element from Each Row	8
7	Further Reading	8

1 Lecture Overview

Lists are the first *workhorse* data structure most Python learners use for storing many values in one variable. This lecture focuses on:

- how to create lists and access elements (indexing and slicing),
- how mutation works (methods that change the list in place),
- how to build new lists using list comprehensions, and
- how to avoid common beginner bugs (aliasing and editing while iterating).

The demo script uses a list of marks to compute summary statistics and shows a realistic use of sorting and comprehensions.

2 Core Concepts

2.1 What Is a List?

A **list** is an **ordered** and **mutable** collection:

- **Ordered** means items have positions (index 0, 1, 2, ...).
- **Mutable** means the contents can change after creation (append, insert, remove, sort, etc.).
- Lists allow duplicates and can store mixed types. In practice, keep a list *conceptually uniform* (e.g., a list of marks is all integers).

2.2 Creating Lists (Common Patterns)

```
numbers = [2, 4, 6] # literal
letters = list("python") # from an iterable (string)
nums = list(range(5)) # from a range
items = ["A", 3.5, True] # mixed types (allowed)
empty = [] # empty list
```

The constructor `list(...)` works with any iterable. For example, `list(range(5))` produces `[0, 1, 2, 3, 4]`.

2.3 Indexing and Negative Indexing

Python uses **zero-based indexing**:

```
scores = [80, 92, 75, 88]
first = scores[0] # 80
last = scores[-1] # 88 (negative index counts from the end)
```

If you try to access an index that does not exist, Python raises an `IndexError`. This is good: it prevents silent wrong answers.

2.4 Slicing (start:stop:step)

Slicing returns a **new list** (a copy of the selected region):

```
nums = [1, 2, 3, 4, 5, 6]
nums[1:4] # [2, 3, 4] (stop index 4 is excluded)
nums[::2] # [1, 3, 5] (step = 2)
nums[-3:] # last three elements
nums[::-1] # reversed copy (step = -1)
```

Typical uses:

- Take the first/last k elements: `data[:3]`, `data[-3:]`.
- Sample regularly (every second item): `data[::2]`.
- Reverse without mutating: `rev = data[::-1]`.
- Create a shallow copy: `copy = a[:]`.

2.5 List Methods: Mutating vs Returning New

Many list methods **mutate** (modify) the same list object:

Method	Effect
<code>append(x)</code>	add one item at the end
<code>extend(iter)</code>	add many items from an iterable
<code>insert(i, x)</code>	insert at a position
<code>remove(x)</code>	remove first occurrence of a value
<code>pop(i)</code>	remove and return item at index
<code>clear()</code>	remove all items (empty the list)
<code>sort()</code>	sort the list in place
<code>reverse()</code>	reverse the list in place

In contrast, `sorted(...)` is a **function** that returns a **new list**:

```
nums = [3, 1, 2]
nums.sort() # nums becomes [1, 2, 3]
new_nums = sorted(nums, reverse=True) # returns a new list
```

Rule of thumb: if you need the original ordering later, prefer `sorted`. If you are done with the old order, `sort()` is fine.

Small Notes on Common Methods

- `remove(x)` removes by **value** (first match) and raises `ValueError` if `x` is not present.
- `pop(i)` removes by **index** and returns the removed item. With no argument, `pop()` removes the last element.
- `reverse()` mutates the list. If you want a reversed *copy*, use slicing: `rev = a[::-1]`.

2.6 List Comprehensions (Filter + Transform)

Comprehensions let you build a new list concisely:

```
squares = [x*x for x in range(6)]  
even_squares = [n*n for n in nums if n % 2 == 0]
```

Reading tip: say it in English.

“Take $n * n$ for each n in `nums`, if n is even.”

If a comprehension becomes hard to read (multiple conditions, nested loops), switch to a normal `for` loop for clarity.

Filtering vs if-else in a Comprehension

Two patterns look similar but do different things:

- **Filter (keep/remove elements):** `[x for x in data if cond]`
- **Choose a value (same length output):** `[A if cond else B for x in data]`

Example:

```
nums = [1, 2, 3, 4, 5, 6]  
evens = [n for n in nums if n % 2 == 0] # [2, 4, 6]  
labels = ["even" if n % 2 == 0 else "odd" for n in nums] # ["odd", "even", ...]
```

Nested Comprehensions (Flattening 2D Data)

When you have a list of lists, nested comprehensions can flatten data:

```
matrix = [[1, 2, 3], [4, 5, 6]]  
flat = [x for row in matrix for x in row]
```

This is equivalent to two nested `for` loops.

2.7 Nested Lists (2D Data)

A nested list is a list that contains other lists. A common pattern is a 2D matrix or table:

```
matrix = [[1, 2, 3], [4, 5, 6]]  
matrix[1][2] # 6 (row 1, col 2)
```

Important pitfall: avoid creating a matrix with list multiplication like `[[0]*3]*2`. That creates aliases of the same inner list. Prefer:

```
rows, cols = 2, 3  
matrix = [[0 for _ in range(cols)] for _ in range(rows)]
```

3 Common Pitfalls (and How to Avoid Them)

3.1 Aliasing vs Copy

Assignment does **not** copy a list; it creates a new name pointing to the same list object:

```
a = [1, 2, 3]
b = a # alias (same object)
b.append(99)
a # [1, 2, 3, 99]
```

To create an independent list:

```
b = a.copy() # or: b = a[:]
```

3.2 Modifying While Iterating

Removing items while iterating can skip elements because indices shift. Two safe patterns are:

- build a new list using a filter (comprehension), or
- iterate over a copy (`for x in a[:]`) when appropriate.

4 Demo Walkthrough: Student Marks Analyzer

Script: `demo/list_marks_analyzer.py`

4.1 What the Demo Is Teaching

The demo ties together list fundamentals in a realistic task:

- compute summary values (average),
- rank values (top-3 using sorting and slicing),
- filter values (passing marks using a comprehension).

4.2 Suggested Run Steps

```
python demo/list_marks_analyzer.py
```

Ask students to predict outputs *before* running the script. Prediction forces them to reason about list operations, not just watch results.

4.3 Extensions (Optional)

- Compute the median mark (hint: sort a copy first).
- Count pass/fail and print percentages.
- Print the top-3 marks with ranks (1st, 2nd, 3rd).

5 Quiz and Solutions

This section contains the interactive checkpoints from the lecture, plus clear solutions.

5.1 Checkpoint 1 (Mutating vs Returning New)

Question. Which of these returns a new list instead of mutating? `append`, `sort`, `sorted`.

Solution. `sorted` returns a *new* list. Both `append` and `sort` mutate the existing list.

5.2 Checkpoint 2 (Comprehension)

Question. Write a list comprehension to square only even numbers from `nums = [1,2,3,4,5,6]`

Solution.

```
nums = [1, 2, 3, 4, 5, 6]
result = [n*n for n in nums if n % 2 == 0]
# result = [4, 16, 36]
```

5.3 Checkpoint 3 (Filtering vs if-else)

Question. What is the difference between these two comprehensions?

1. `[x for x in data if cond]`
2. `[A if cond else B for x in data]`

Solution.

- The first is a **filter**: elements that do not satisfy `cond` are removed, so the output may be shorter.
- The second is a **value choice**: you keep one output element per input element, so output length stays the same.

Example:

```
nums = [1, 2, 3, 4]
only_even = [n for n in nums if n % 2 == 0] # [2, 4]
labels = ["E" if n % 2 == 0 else "O" for n in nums] # ["O", "E", "O", "E"]
```

5.4 Think-Pair-Share (Readability)

Prompt. When is a **for-loop** clearer than a list comprehension?

Sample response. A loop is clearer when the logic has multiple steps, needs debugging prints, or has side effects. For example, validating inputs and collecting errors is usually clearer as a loop than as a complex comprehension.

5.5 Exit Question (Slicing)

Question. What slice returns the last three items of a list `data`?

Solution. `data[-3:]`.

6 Practice Exercises (With Solutions)

6.1 Exercise 1: Remove Duplicates (Keep Order)

Task. Given data, create a new list that removes duplicates but keeps the first-occurrence order.
Solution.

```
def unique_keep_order(data):  
    seen = set()  
    out = []  
    for x in data:  
        if x not in seen:  
            seen.add(x)  
            out.append(x)  
    return out
```

6.2 Exercise 2: Top-3 Without Mutating the Input

Task. Write a function that returns the top-3 values of a list without changing the original list.
Solution.

```
def top3(values):  
    return sorted(values, reverse=True)[:3]
```

6.3 Exercise 3: 2D Sum

Task. Given a matrix (list of lists), compute the sum of all elements.
Solution.

```
def sum_matrix(matrix):  
    total = 0  
    for row in matrix:  
        for x in row:  
            total += x  
    return total
```

6.4 Exercise 4: Create and Display a List

Task. Create a list of your favorite foods and print it in a readable way.
Solution.

```
foods = ["biryani", "pizza", "mango"]  
print("Foods:", foods)  
for i, item in enumerate(foods, start=1):  
    print(i, item)
```

6.5 Exercise 5: Remove All Even Numbers

Task. Given a list of integers, create a new list that contains only odd numbers (do not modify while iterating).
Solution.

```
nums = [1, 2, 3, 4, 5, 6, 7]
odds = [n for n in nums if n % 2 == 1]
print(odds) # [1, 3, 5, 7]
```

6.6 Exercise 6: Squares from 1 to 10 (Comprehension)

Task. Generate squares from 1 to 10 using a list comprehension.

Solution.

```
squares = [n*n for n in range(1, 11)]
```

6.7 Exercise 7: Second Element from Each Row

Task. Given a nested list (table), print the second element from each row.

Solution.

```
table = [
    ["A", 10, 20],
    ["B", 11, 21],
    ["C", 12, 22],
]
second_col = [row[1] for row in table]
print(second_col) # [10, 11, 12]
```

7 Further Reading

Official Python docs are short and practical:

- <https://docs.python.org/3/tutorial/datastructures.html>

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: <https://github.com/tali7c/Python-Programming>

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Overview
- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Create lists and access elements using indexing and slicing
- Apply core list methods for insertion, deletion, and sorting
- Write list comprehensions for concise transformations
- Explain and use nested list structures

What is a List?

- Ordered collection of items
- Mutable (can change after creation)
- Allows duplicates and mixed types
- Ideal for sequences you need to update

Creating Lists

- Literal: `numbers = [2, 4, 6]`
- Constructor: `letters = list("python")`
- From range: `nums = list(range(5))`
- Mixed types: `items = ["A", 3.5, True]`

Indexing

- Zero-based indexing: first element at index 0
- Negative indexing counts from the end
- Example: `scores = [80, 92, 75, 88]`

```
scores = [80, 92, 75, 88]
print(scores[0]) # 80
print(scores[-1]) # 88
```

Slicing

- Syntax: `list[start:stop:step]`
- Stop index is excluded
- Useful for sublists and stepping

```
nums = [1, 2, 3, 4, 5, 6]
print(nums[1:4]) # [2, 3, 4]
print(nums[::2]) # [1, 3, 5]
print(nums[::-1]) # [6, 5, 4, 3, 2, 1]
```

Core List Methods

Method	Purpose
<code>append(x)</code>	Add to end
<code>extend(iter)</code>	Add all items
<code>insert(i, x)</code>	Insert at index
<code>remove(x)</code>	Remove first match
<code>pop(i)</code>	Remove by index
<code>clear()</code>	Remove all items
<code>reverse()</code>	Reverse in place

- Most methods mutate the list in place

Sorting: `sort` vs `sorted`

- `sort()` modifies the list in place
- `sorted(list)` returns a new list
- Use `sorted` when you must preserve the original

```
nums = [3, 1, 2]
nums.sort() # nums becomes [1, 2, 3]
new_nums = sorted(nums, reverse=True)
```

List Operations

- Concatenation: `[1,2] + [3,4]`
- Repetition: `[0] * 3`
- Membership: `5 in nums`
- Length: `len(nums)`

Pitfall: Aliasing vs Copy

```
a = [1, 2, 3]
b = a # alias (same object)
b.append(99)
print(a) # [1, 2, 3, 99]
```

- Fix: create an independent list copy

```
b = a.copy() # or: a[:]
```

List Comprehension

- Compact way to build lists
- Pattern: `[expr for item in iterable if cond]`

```
squares = [x*x for x in range(6)]
```

Comprehension Example

```
nums = [3, 4, 7, 8, 10]
result = [n*n for n in nums if n % 2 == 0]
print(result) # [16, 64, 100]
```

- Combine filtering and transformation in one line

Comprehensions: Filtering vs if-else

- Filter (keep/remove elements): `[x for x in data if cond]`
- Choose a value (same length): `[A if cond else B for x in data]`

```
nums = [1, 2, 3, 4, 5, 6]
evens = [n for n in nums if n % 2 == 0]
labels = ["even" if n % 2 == 0 else "odd" for n in nums]
```

Nested Comprehensions (Flattening)

- Useful for 2D data (lists of lists)
- Reads like two nested loops, left to right

```
matrix = [[1, 2, 3], [4, 5, 6]]
flat = [x for row in matrix for x in row]
print(flat) # [1, 2, 3, 4, 5, 6]
```

Worked Example: Marks Analyzer (Core Steps)

```
marks = [72, 88, 95, 64, 51, 43, 77]

top_three = sorted(marks, reverse=True)[:3]
pass_mark = 50
passing = [m for m in marks if m >= pass_mark]
```

- Use `sorted` to avoid mutating the input list
- Use a comprehension to filter by a rule

Nested Lists

- Lists can contain other lists (2D structures)
- Access with two indices: `matrix[row][col]`

```
matrix = [[1, 2, 3], [4, 5, 6]]
print(matrix[1][2]) # 6
```

Pitfall: Modify While Iterating

- Removing items while iterating can skip elements
- Prefer building a new list (filter) or iterate over a copy

```
nums = [1, 2, 3, 4, 5]
nums = [n for n in nums if n % 2 == 1]
print(nums) # [1, 3, 5]
```

Demo: Student Marks Analyzer

- Load a list of marks
- Compute average and top 3 scores
- Filter passing marks using a comprehension
- Extension: compute median as a challenge

Script: `demo/list_marks_analyzer.py`

Checkpoint 1

Which of these returns a new list instead of mutating?

1. `append`
2. `sort`
3. `sorted`

Checkpoint 2

Write a list comprehension to square only even numbers from `nums = [1,2,3,4,5,6]`.

Checkpoint 3

What is the difference between these two comprehensions?

1. `[x for x in data if cond]`
2. `[A if cond else B for x in data]`

Think-Pair-Share

When is a **for-loop** clearer than a list comprehension? Give a short example and justify your choice.

Summary

- Lists are ordered, mutable collections
- Indexing/slicing enable flexible access
- Methods like `append` and `sort` mutate lists
- Comprehensions build lists concisely

Exit Question

What slice returns the last three items of a list `data`?